

Distributed Processing of Credit-Card Transactions

1. Requirements

Users:

- Fraud analysts: Monitor suspicious transactions.
- Automated system: Checks transactions in real-time.

Functional Requirements:

- Real-time check: Detect if a transaction involves a known fraudulent entity.
- Batch processing: Discover new fraud patterns overnight.
- Graph storage: Transactions as nodes (entities) and edges (transactions).

Non-functional Requirements:

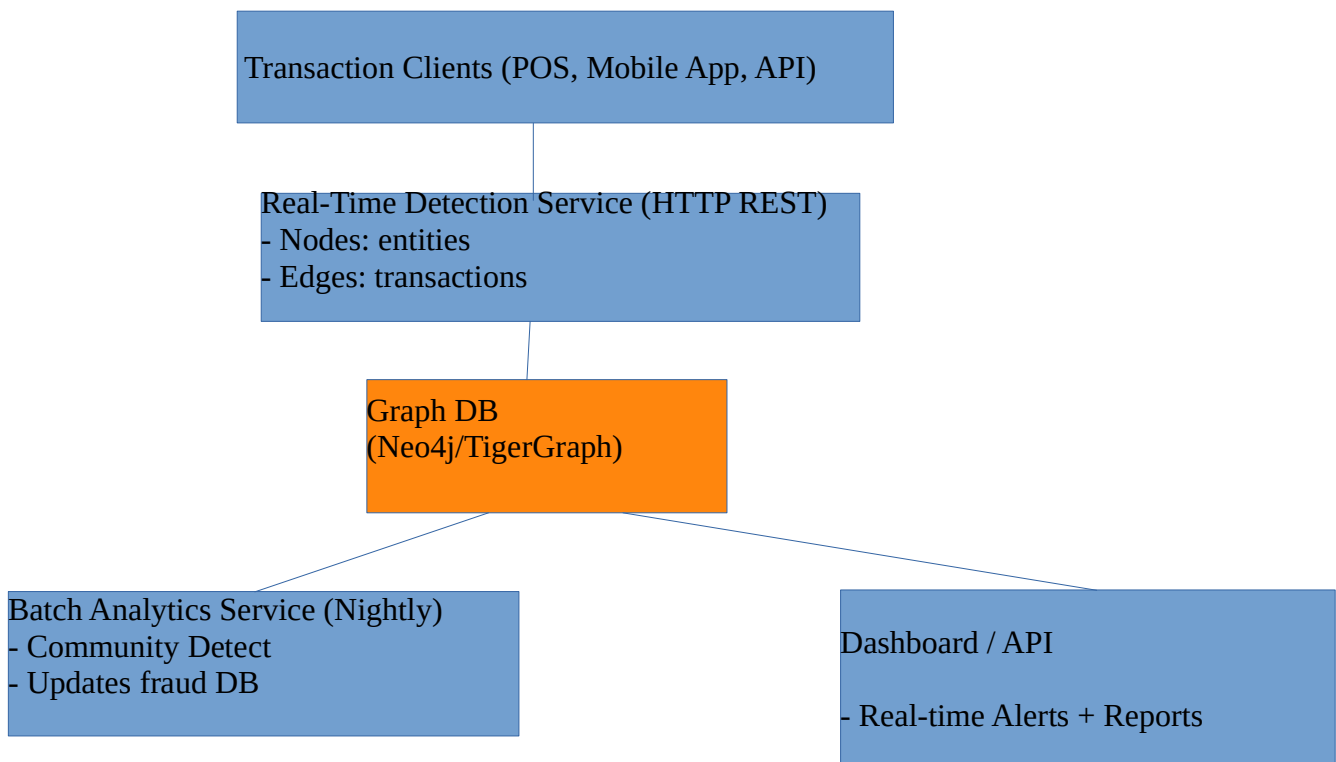
- Real-time latency must be smaller than 1 second per transaction
- Scalable to millions of transactions per day
- Fault-tolerant and secure

2. System Architecture

Components:

1. **Transaction API (HTTP POST):** Clients send transactions.
2. **Real-Time Detection Service:**
 - Checks new transactions against known fraud using graph traversal (BFS).
 - Returns “alert” or “safe” instantly.
3. **Graph Database (Neo4j/TigerGraph):** Stores entities and transactions.
4. **Batch Analytics Service:**
 - Runs overnight to find new fraud patterns using PageRank or community detection.
 - Updates fraud entity list for real-time checks.
5. **Dashboard/API Service:** Shows alerts and reports for analysts.

Architecture Diagram:



Distributed Techniques:

- Graph partitioning for scalability
- Replication for fault tolerance
- Separate real-time and batch services for performance

3. System Evaluation

Advantages:

- Simple implementation : it uses familiar REST APIs, avoiding the need for complex message queues, making it easier to develop and maintain.

Challenges:

- Scalability: Large graph data with millions of transactions.
- Real-time processing: Low-latency graph queries are hard.
- Fault tolerance: System must survive node failures.
- Consistency: Real-time checks need fresh data; batch updates can be slightly delayed.
- Complex graph algorithms: BFS, PageRank, and community detection are resource-intensive.

Freestyle Exercise: Instantiate a Particular DS Concept

1. Concept / Enhancement

The system extends the basic messenger protocol with:

- **Reliable user identification:** The server maps UDP packets to registered clients to prevent spoofing.
- **Online presence detection:** Clients send a heartbeat every 60 seconds; clients are removed after 120 seconds of inactivity.
- **System notifications:** Messages are broadcast when clients join or leave the system.
- **Online user listing:** Clients can query currently active users using the 'online' command.
- **Message routing:** Supports unicast and broadcast while preserving the sender identity.

This design enhances fault-tolerance, state consistency, and user awareness in a distributed messaging system.

2. Implementation

- **Client:**
 - Registers with the server via TCP; sends periodic UDP heartbeats.
 - Receives messages via UDP.
 - Supports commands: <targetId> <message> for unicast, 0 <message> for broadcast, and online to list active users.
- **Server:**
 - Maintains a `ConcurrentHashMap<UserIdentifier, ClientInfo>` to track online clients.
 - Identifies message senders by matching UDP packet source to registered clients.
 - Routes messages correctly (unicast or broadcast) and responds to online queries.
 - Removes clients that do not send a heartbeat within the timeout, broadcasting leave notifications.
- **Reliability / Consistency:**
 - Heartbeats ensure timely detection of offline clients.
 - Sender identity is verified to maintain consistent and reliable message delivery.

Windows PowerShell

PS Microsoft.PowerShell.Core\FileSystem::\\WSL.LOCALHOST\Ubuntu-24.04\home\youb\working\distributed-systems-2025-ayoubG23\src> javac de/luh/vss/chat/common/*.java de/luh/vss/chat/client/*.java de/luh/vss/chat/server/*.java
PS Microsoft.PowerShell.Core\FileSystem::\\WSL.LOCALHOST\Ubuntu-24.04\home\youb\working\distributed-systems-2025-ayoubG23\src> java de.luh.vss.chat.server.Server
Server started on port 5000

Server

Windows PowerShell

PS Microsoft.PowerShell.Core\FileSystem::\\WSL.LOCALHOST\Ubuntu-24.04\home\youb\working\distributed-systems-2025-ayoubG23\src> java de.luh.vss.chat.client.ChatClient 123
Client started as user 123
UDP port: 52779
>
[from 0] [SYSTEM] User 123 joined
>
[from 0] [SYSTEM] User 456 joined
>
[from 0] [SYSTEM] User 789 joined
>
[from 0] [SYSTEM] User 2 joined
message
>
[from 0] [SYSTEM] User 789 left
>

Client

Windows PowerShell

PS Microsoft.PowerShell.Core\FileSystem::\\WSL.LOCALHOST\Ubuntu-24.04\home\youb\working\distributed-systems-2025-ayoubG23\src> java de.luh.vss.chat.client.ChatClient 456
Client started as user 456
UDP port: 52780
>
[from 0] [SYSTEM] User 456 joined
>
[from 0] [SYSTEM] User 789 joined
>
[from 789] i am broadcasting a message
> online
>
[from 0] Online users: 2, 456, 123
>
>

Client

Windows PowerShell

me\youb\working\distributed-systems-2025-ayoubG23\src> java de.luh.vss.chat.client.ChatClient 789
Client started as user 789
UDP port: 52781
>
[from 0] [SYSTEM] User 789 joined
> > online
>
[from 0] Online users: 789, 456, 123
> 67 i am sending to an invalide User !
>
[from 0] [SYSTEM] Unknown recipient: 67
> 0 i am broadcasting a message
> exit
Client terminated.
PS Microsoft.PowerShell.Core\FileSystem::\\WSL.LOCALHOST> java de.luh.vss.chat.client.ChatClient 2
Client started as user 2
UDP port: 58320
>
[from 0] [SYSTEM] User 2 joined
> >

Client

3. Evaluation

Aspect	Analysis
Fault-tolerance	Heartbeat mechanism detects inactive or crashed clients. Messages to offline clients are dropped and the sender recieve feedback.
Consistency	Sender verification and the authoritative server state ensure messages are correctly attributed.
Scalability	ConcurrentHashMap and UDP allow handling multiple clients efficiently. TCP is only used for registration.
Performance	UDP delivery provides low latency. TCP overhead is minimal.
Limitations	UDP delivery: UDP does not guarantee message delivery, so some messages may be lost. This trade-off was chosen to reduce latency and overhead, prioritizing real-time communication. NAT / address mapping: In some network setups, NAT may cause mismatches between the reported and actual sender addresses. The system works reliably in standard local network configurations. Offline buffering: Messages are not buffered for offline users, because clients that do not send heartbeats are considered offline. This simplifies the system and reduces memory and processing overhead, while keeping the focus on active participants.

4. Potential Improvements

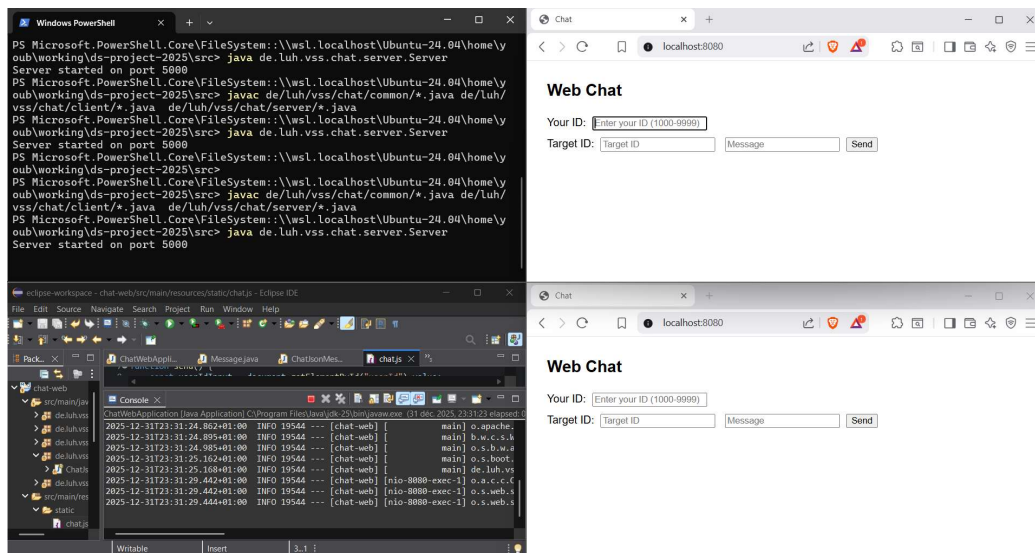
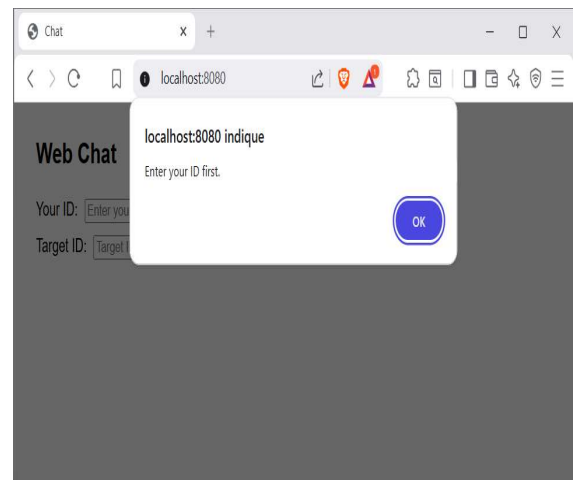
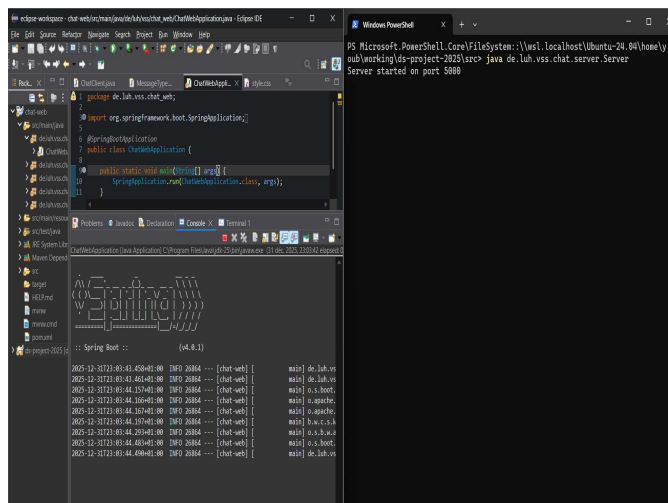
Implement message buffering and retransmission to handle UDP packet loss.

Introduce persistent storage to retain client states across server restarts.

Maintain message ordering for broadcasts using sequence numbers.

scaling with multiple servers and replication for load balancing.

Scalability and framework integration: A real-world system could leverage a framework such as Spring Boot to handle multiple servers, replication, and load balancing. An initial exploration of this approach was conducted (see screenshot), but the complexity of full integration and due to time shortage i couldnt .



5. Conclusion

The system demonstrates an enhanced distributed messaging protocol that incorporates:

- Reliable sender verification.

- Automatic detection of online/offline clients.
- System notifications for joins and leaves.
- Online user queries and broadcast/unicast messaging.

It provides a practical implementation of distributed system concepts such as **consistency, fault detection, and scalability**, and forms a foundation for further enhancements.